

# RAGMED — Sistema de Recuperación y Generación Aumentada para Diagnóstico de Enfermedades

---

Sistema RAG que, dado un conjunto de síntomas, sugiere posibles enfermedades usando Wikipedia como corpus y un LLM local servido mediante Ollama.

---

## Requisitos

- **Python 3.11+**
  - [uv](#) (gestor de paquetes recomendado)
  - [Ollama](#) instalado y en ejecución
- 

## Instalación

### 1. Instalar uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

O con pip:

```
pip install uv
```

### 2. Instalar dependencias Python

Desde la raíz del repositorio (donde se encuentra `pyproject.toml`):

```
uv sync
```

Alternativa con pip estándar (sin uv):

```
pip install ollama requests beautifulsoup4 rank-bm25 lxml gradio
```

### 3. Descargar modelos de Ollama

```
ollama pull hf.co/CompendiumLabs/bge-base-en-v1.5-gguf
ollama pull hf.co/bartowski/Llama-3.2-1B-Instruct-GGUF
```

Asegúrate de que el servicio Ollama está activo antes de ejecutar el sistema (`ollama serve` si no arranca automáticamente).

---

## Ejecutar el sistema

El corpus `diseases.txt` está incluido en este paquete. Si ya existe, puedes omitir el crawler con `--skip-crawler`.

## Interfaz de línea de comandos (CLI)

Desde la raíz del repositorio:

```
# Usar el corpus incluido directamente (recomendado):
uv run python Sistema/raged_main.py --skip-crawler

# Regenerar el corpus completo A-Z (tarda varios minutos por el límite de velocidad de
  Wikipedia):
uv run python Sistema/raged_main.py

# Muestra aleatoria de 50 enfermedades de todo el alfabeto:
uv run python Sistema/raged_main.py --max-diseases 50 --shuffle

# Prueba rápida – 20 enfermedades aleatorias, solo letras A y B:
uv run python Sistema/raged_main.py --max-diseases 20 --letters A B --shuffle
```

## Opciones de línea de comandos

| Flag                            | Por defecto                   | Descripción                                                                                                     |
|---------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>--skip-crawler</code>     | desactivado                   | Omite la descarga y usa el corpus existente                                                                     |
| <code>--max-diseases N</code>   | todas                         | Limita el número de enfermedades a descargar                                                                    |
| <code>--letters A B ...</code>  | A–Z                           | Restringe la descarga a esas letras del índice                                                                  |
| <code>--shuffle</code>          | desactivado                   | Aleatoriza la lista antes de aplicar <code>--max-diseases</code>                                                |
| <code>--similarity-fn</code>    | hybrid                        | Función de similitud: <code>hybrid</code> , <code>cosine</code> , <code>euclidean</code> , <code>jaccard</code> |
| <code>--alpha</code>            | 0.5                           | Peso del componente coseno en modo híbrido                                                                      |
| <code>--top-n</code>            | 5                             | Número de fragmentos recuperados por consulta                                                                   |
| <code>--corpus-file PATH</code> | <code>diseases.txt</code>     | Ruta del corpus consolidado                                                                                     |
| <code>--list-file PATH</code>   | <code>disease_list.txt</code> | Ruta de la lista de nombres de enfermedades                                                                     |

## Interfaz web

El sistema incluye una interfaz web basada en Gradio:

```
uv run python Sistema/raged_web.py
```

Abre el navegador en **http://localhost:7860**.

La primera vez que se lanza, el sistema embebe todos los fragmentos del corpus (puede tardar varios minutos). Las ejecuciones siguientes cargan los embeddings desde caché (`diseases.txt.cache.pkl`) y arrancan en segundos.

La interfaz incluye: - **Chat** — escribe tus síntomas y recibe un diagnóstico diferencial generado por el LLM. - 🔍 **Retrieved context** — acordeón oculto; expándelo para ver exactamente qué fragmentos del corpus se usaron para generar la respuesta, con sus puntuaciones de similitud. - ⚙️ **Configuration** —

ajusta la función de recuperación (`hybrid`, `cosine`, `euclidean`, `jaccard`), el peso  $\alpha$  del componente coseno y el número de fragmentos recuperados (top-N).

---

## Estructura de ficheros

```
(raíz del paquete)
diseases.txt          # Corpus consolidado (~20 MB, incluido en el paquete)
Sistema/
  _helpers.py         # Utilidades compartidas (parse_sections, find_section, safe_filename)
  ragmed_crawler.py   # Módulo de adquisición de datos (Wikipedia)
  ragmed_source.py    # Implementación WikipediaDiseaseSource
  ragmed_rag.py       # Módulo de recuperación + generación (BM25 híbrido + Ollama)
  ragmed_main.py      # Punto de entrada CLI
  ragmed_web.py       # Interfaz web (Gradio)
  ragmed_eval.py      # Script de evaluación por lotes
  similarity/         # Funciones de similitud (cosine, euclidean, jaccard, hybrid)
  tests/              # Tests unitarios (no requieren red ni Ollama)
```

---

## Ejecutar los tests

El proyecto incluye una suite de tests unitarios que no requieren red ni Ollama:

```
uv run pytest
# Con salida detallada:
uv run pytest -v
```

Los tests cubren: - `_helpers.py`: `safe_filename`, `parse_sections`, `find_section` -  
`ragmed_crawler.py`: pipeline completo con un `InMemoryDiseaseSource` (sin peticiones HTTP)